Nama : Mohamad Zaky Bahtiar Arifianto **Tugas Pertemuan 14**
Matkul : Praktikum Pemrogramman Berorientasi Obyek
Jurusan : Teknik Informatika
NRP : 3125522015

BAB I: PENETAPAN KONDISI AWAL & DESAIN PENGUJIAN

Sebelum skenario pengujian fungsional dan pengujian regresi dijalankan, kondisi awal (baseline state) sistem ditetapkan secara cermat untuk memastikan lingkungan pengujian bersih, deterministik, dan dapat diulang (repeatable).

1. Berkas Data Utama: mbg_data.txt
2. Berkas Cadangan (Backup): mbg_data_backup.txt
3. Jumlah Data Awal: 2 (dua) rekaman paket menu baku
4. Format Karakter Pemisah: Tanda pipa vertikal ('|')

Data Awal Baku:
• MNU-01|Paket A: Nasi Ayam Teriyaki & Tumis Sayur|680.0|28.5|82.0|15000.0
• MNU-02|Paket B: Nasi Telur Balado & Sayur Sop|640.0|24.0|79.0|14000.0

BAB II: TABEL EKSEKUSI SKENARIO PENGUJIAN SISTEM MBG

Sepuluh skenario pengujian fungsional spesifik domain Program Makan Bergizi Gratis (MBG) telah dieksekusi secara otomatis maupun manual dengan hasil kelulusan 100%:

No.	Skenario Pengujian	Hasil yang Diharapkan	Hasil Aktual
1	Tampilkan collection kosong (Opsi 2 saat koleksi 0 item)	Pesan koleksi kosong; aplikasi tetap berjalan stabil.	Muncul: "Koleksi menu MBG masih kosong."; loop menu aktif. [PASSED]
2	Tambah kode baru dengan judul valid (MNU-03)	Data bertambah satu; status dirty flag aktif; total menjadi 3.	Menu berhasil ditambahkan; status ada perubahan aktif. [PASSED]
3	Tambah kode yang sudah terdaftar (Duplikasi MNU-01)	Ditolak secara fail-fast; koleksi data lama tidak tertimpa.	Ditampilkan: "Data ditolak: Kode menu 'MNU-01' sudah terdaftar." [PASSED]
4	Tambah kode atau judul spasi saja (Kode/nama spasi kosong)	Ditolak oleh konstruktor objek domain; tidak masuk koleksi.	Muncul pesan penolakan: "Kode/Nama menu tidak boleh kosong." [PASSED]
5	Tambah judul mengandung '|' (Karakter pemisah terlarang)	Ditolak sebelum masuk collection (Class Invariant).	Muncul: "Nama menu tidak boleh mengandung pemisah '|...". [PASSED]
6	Cari kode dengan huruf kecil (Input: mnu-01)	Menu ditemukan (case-insensitive lookup); rincian tercetak.	Menu MNU-01 berhasil ditemukan dan rincian gizi tercetak lengkap. [PASSED]
7	Hapus kode yang tidak terdaftar (Input: MNU-999)	Pesan tidak ditemukan tercetak; koleksi tetap utuh.	Muncul pesan: "Menu tidak ditemukan." Koleksi tetap utuh. [PASSED]
8	Simpan, tutup, lalu jalankan kembali	Identitas dan isi data tetap sesuai;	Data MNU-05 tersimpan

	(Persistensi MNU-05)	auto-load saat startup.	permanen di disk dan termuat saat startup. [PASSED]
9	Muat file dengan baris rusak/anomali (Uji State Invariance)	Ditolak dengan IOException; state di memori tetap utuh.	Ditolak: "Format salah pada baris 1"; koleksi memori tetap 2 data. [PASSED]
10	Keluar saat ada perubahan, lalu batalkan (Opsi 0 -> jawab TIDAK)	Aplikasi kembali ke menu; data perubahan tetap tersedia.	Muncul: "Batal keluar..."; data perubahan tetap aman di memori. [PASSED]

BAB III: CATATAN REFACTORING & PERBAIKAN BUG

3.1 Tabel Temuan Bug (Defect Tracking Log):

ID Bug	Langkah Pemicu	Hasil Diharapkan	Hasil Aktual Awal	Solusi & Status
BUG-01	Input kode/nama berupa spasi kosong " "	Ditolak saat instansiasi objek.	String spasi lolos jika hanya cek .isEmpty().	Gunakan .trim().isEmpty() pada setter Menu.java. [RESOLVED]
BUG-02	Input nama mengandung delimiter "|"	Ditolak sebelum masuk ke koleksi memori.	Lolos ke RAM, baru meledak saat simpan berkas.	Pindahkan validasi pipa ke konstruktor Menu.java. [RESOLVED]
BUG-03	Cari kode dengan huruf kecil (mnu-01)	Menu ditemukan tanpa sensitivitas kapital.	Pencarian gagal karena pembandingan case-sensitive.	Gunakan .equalsIgnoreCase(kode.trim()) di PengelolaMenu. [RESOLVED]

3.2 Tiga Catatan Refactoring Struktur Kode (Clean Code):

1. Dekomposisi Switch-Case Menu di Main.java (Extract Method Pattern):
Seluruh blok menu panjang pada main loop dipecah menjadi method-method private yang independen (menuTambah, menuTampilkan, menuCari, menuHapus, menuSimpan, menuMuatUlang, menuKeluar). Alasan: Menegakkan Single Responsibility Principle (SRP) dan Separation of Concerns (SoC) sehingga method utama menjadi dispatcher ringkas satu baris.

2. Penegakan Aturan Class Invariant pada Objek Domain & Pengelola:
Seluruh validasi format masukan (spasi kosong, tanda pemisah pipa, batasan angka gizi) dipusatkan secara mutlak di dalam konstruktor dan setter Menu.java. Sementara aturan keunikan kode menu dipusatkan di PengelolaMenu.java. Alasan: Mencegah objek cacat hidup di heap memory (invalid state prevention) dan menjamin aturan bisnis tetap ditegakkan terlepas dari antarmuka apa pun yang mengaksesnya.

3. Penerapan Prinsip DRY (Don't Repeat Yourself) pada Input Konsol:
Mengekstrak helper method baca(Scanner, String) dan parseDouble(String, String) untuk menghindari penulisan ulang Scanner dan try-catch konversi angka, sekaligus mengatasi newline residue bug.

BAB IV: JAWABAN EVALUASI RESMI PRAKTIKUM P14

1. Pertanyaan 1: Mengapa program yang berhasil dikompilasi masih perlu diuji?
Jawaban: Kompilasi (javac) hanya menjamin keabsahan sintaksis dan kesesuaian tipe data statis. Kompilasi tidak mampu mendeteksi kesalahan logika bisnis, pelanggaran invariant runtime, penanganan input anomali, maupun kegagalan sistem berkas. Pengujian empiris mutlak diperlukan guna membuktikan kesesuaian fungsi aplikasi dengan kebutuhan pengguna.

2. Pertanyaan 2: Apa perbedaan perbaikan bug dan refactoring?
Jawaban: Perbaikan Bug (Bug Fixing) berfokus pada mengubah perilaku program dari yang semula salah menjadi benar sehingga luaran berubah. Sebaliknya, Refactoring berfokus pada memperbaiki struktur internal kode (keterbacaan, modularitas, DRY) TANPA MENGUBAH sedikit pun perilaku eksternal (observable behavior) yang sudah berjalan benar.

3. Pertanyaan 3: Mengapa pengujian harus diulang setelah struktur kode diubah (uji regresi)?
Jawaban: Pengubahan struktur kode berisiko menimbulkan efek samping yang tidak disengaja (side effects). Pengujian regresi dilakukan untuk memverifikasi secara empiris bahwa refactoring tidak merusak fitur-fitur stabil yang telah dibangun sebelumnya.

4. Pertanyaan 4: Bagaimana membuktikan operasi yang gagal (misal file corrupt) tidak merusak state objek di memori?
Jawaban: Dibuktikan melalui mekanisme staging/buffer sementara (seperti di MbgFileRepository.muat()). Koleksi aktif di memori dicatat sebelum operasi. Saat berkas korup dimuat, eksepsi fail-fast terjadi pada koleksi sementara. Koleksi utama di memori tidak diubah sedikit pun, sehingga jumlah dan integritas objek tetap utuh (State Invariance terbukti).

BAB V: KESIMPULAN

Praktikum 14 berhasil menyempurnakan Sistem Manajemen MBG menjadi aplikasi yang tangguh, modular, dan handal dengan tingkat kelulusan uji 100%. Kode sumber telah bersih dari redundansi, memiliki batas tanggung jawab yang tegas antar-kelas, serta dilengkapi dokumentasi teknis dan panduan operasional demonstrasi.